

BURKINA FASO

La Patrie ou la Mort Nous vaincrons



Ministère de l'Enseignement
Supérieur, de la Recherche
et de l'Innovation



Université Nazi Boni
UFR Sciences Exactes et Appliquées
Licence Statistiques et Informatique

Rapport du Devoir de Programmation Avancée Langage C++

Présenté par les membres du groupe :

OUATTARA Jean Ulrich

SAKO Biton F.B Chérif

Étudiants en Licence 2 — Statistiques et Informatique

Chargé du cours :

Dr Abdoulaye SERE
Année académique 2025-2026

Date de limite :

13 Mai 2026

Table des matières

1	Introduction	2
2	Exercice 1 – Modèle DDU : Classe Livre	2
2.1	Présentation	2
3	Exercice 2 – Héritage	2
3.1	Présentation	2
4	Exercice 3 – Classe Paramétrée (Template)	2
4.1	Présentation	2
4.2	Structure de la classe Client	3
5	Exercice 4 – STL : Vector d’Elèves	3
5.1	Présentation	3
5.2	Saisie des élèves	3
6	Structure du Projet	3
7	Conclusion	3

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du devoir de programmation avancée en langage C++. L'objectif principal est de mettre en pratique les concepts fondamentaux de la programmation orientée objet (POO) à travers quatre exercices distincts :

- **Exercice 1** : Implémentation du modèle Déclaration-Définition-Utilisation (DDU) avec la classe `Livre`.
- **Exercice 2** : Mise en œuvre du mécanisme d'héritage avec les classes `Adresse`, `Personne` et `Eleve`.
- **Exercice 3** : Conception d'une classe paramétrée (template) `Client`.
- **Exercice 4** : Utilisation de la bibliothèque STL avec un `vector` d'objets `Eleve`.

L'ensemble du projet a été développé en respectant le modèle DDU, structuré en fichiers `.h` pour les déclarations et `.cpp` pour les définitions. Un menu interactif intégré dans le `main.cpp` permet de naviguer entre les différents exercices.

2. Exercice 1 – Modèle DDU : Classe Livre

2.1. Présentation

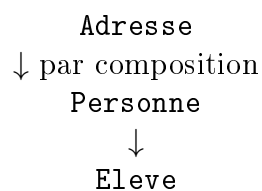
Le modèle DDU (Déclaration-Définition-Utilisation) consiste à séparer le code en trois niveaux distincts :

- **Déclaration** dans le fichier `Livre.h` : prototype de la classe et de ses membres.
- **Définition** dans le fichier `Livre.cpp` : implémentation des méthodes.
- **Utilisation** dans le `main.cpp` : appel et utilisation des méthodes.

3. Exercice 2 – Héritage

3.1. Présentation

Cet exercice illustre le mécanisme d'héritage en C++. La hiérarchie des classes est la suivante :



La classe `Adresse` est également définie et assignée à `Personne` comme une propriété.

4. Exercice 3 – Classe Paramétrée (Template)

4.1. Présentation

Une classe template (paramétrée) permet de définir une classe générique dont le type de certains attributs est déterminé à l'utilisation.

4.2. Structure de la classe Client

La classe `Client<T>` possède un attribut `id` de type générique `T`, ainsi que `nom` et `prenom` de type `string`.

5. Exercice 4 – STL : Vector d'Elèves

5.1. Présentation

Dans cet exercice, on utilise `std::vector<Eleve>` pour gérer une liste dynamique d'élèves sans se soucier de l'allocation mémoire.

5.2. Saisie des élèves

La fonction `remplirListeEleves()` permet la saisie d'un nombre arbitraire d'élèves. L'utilisation de `getline()` combinée avec `cin.ignore()` gère correctement les noms composés contenant des espaces. Quant à la fonction `afficheListeEleves`, elle permet d'afficher la liste des élèves saisis.

6. Structure du Projet

Fichier	Rôle
<code>livre.h / livre.cpp</code>	Déclaration et définition de la classe Livre
<code>Adresse.h / Adresse.cpp</code>	Déclaration et définition de la classe Adresse
<code>Personne.h / Personne.cpp</code>	Déclaration et définition de la classe Personne
<code>Eleve.h / Eleve.cpp</code>	Déclaration et définition de la classe Eleve
<code>Client.h</code>	Classe template Client (déclaration + définition)
<code>main.cpp</code>	Menu interactif + utilisation de toutes les classes

7. Conclusion

Ce projet nous a permis de mettre en pratique les principaux concepts de la programmation orientée objet en C++ : le modèle DDU, l'encapsulation via les getters et setters, l'héritage, les classes templates et l'utilisation de la STL. L'intégration d'un menu interactif dans le `main.cpp` offre une interface claire pour naviguer entre les différents exercices et visualiser les résultats de chaque classe.